



安徽三联学院
ANHUISANLIAN UNIVERSITY

本科毕业设计（论文、创作）

题 目：_____ 基于 A* 算法的智能贪吃蛇游戏设计与实现 _____

学生姓名：_____ 学号：_____

所在学院：_____ 计算机科学与技术学院 _____ 专业：_____ 计算机科学与技术 _____

入学时间：_____ 年 _____ 月

导师姓名：_____ 职称/学位 _____

导师所在单位：_____ 安徽三联学院 _____

完成时间：_____ 2026 _____ 年 _____ 6 _____ 月

安徽三联学院教务处 制

基于 A* 算法的智能贪吃蛇游戏设计与实现

摘要：贪吃蛇游戏自诞生以来便是一款经久不衰的经典小游戏，其规则简单但策略空间丰富，适合用来验证各类搜索算法的实际效果。本文基于 Python 语言和 Pygame 图形库，设计并实现了一款具备自动寻路能力的智能贪吃蛇游戏。系统的核心在于利用 A* 算法驱动蛇的路径规划，使蛇能够在避开自身蛇身与障碍物的前提下，持续追踪食物位置并做出移动决策。论文从 A* 算法的基本原理出发，分析了曼哈顿距离、欧几里得距离等启发式函数对搜索效率和路径质量的影响，并将其与 BFS、Dijkstra 等经典搜索算法进行了性能对比。在实际调试过程中发现，启发式函数的权重选择对搜索结果有显著影响——权重过大会导致搜索不够彻底，错过最优路径；权重过小则退化为类似 Dijkstra 的遍历，计算开销陡增。此外，当蛇身变长导致可用空间急剧缩减时，单纯的 A* 寻路往往无法避免“自己堵死自己”的死局，需要引入安全区域评估等辅助策略。本系统实现了手动操控与 AI 自动运行两种模式，支持游戏界面可视化、分数统计、速度调节等功能。测试结果表明，在中等规模网格下 A* 算法能够以较低的搜索代价完成绝大多数场景的路径规划，但在高密度蛇身条件下仍存在决策失误的情况，这也是后续优化的方向。

关键词：A* 算法；贪吃蛇游戏；路径规划；Pygame；游戏 AI

Design and Implementation of an Intelligent Snake Game Based on A* Algorithm

Abstract: The snake game has been a classic arcade game since its inception, featuring simple rules yet offering rich strategic depth, making it an excellent testbed for various search algorithms. This thesis designs and implements an intelligent snake game with automatic pathfinding capability, built upon the Python programming language and the Pygame graphical library. The core of the system lies in employing the A* algorithm to drive path planning for the snake, enabling it to continuously track food positions and make movement decisions while avoiding collisions with its own body and obstacles. Starting from the fundamental principles of the A* algorithm, the thesis analyzes the impact of heuristic functions such as Manhattan distance and Euclidean distance on search efficiency and path quality, and compares their performance against classical search algorithms including BFS and Dijkstra. During practical debugging, it was found that the weight selection of heuristic functions significantly affects search results — excessively large weights lead to insufficient search and suboptimal paths, while overly small weights degenerate into Dijkstra-like traversal with sharply increased computational overhead. Furthermore, when the snake body grows and available space diminishes, pure A* pathfinding often fails to prevent self-blockage, necessitating auxiliary strategies such as safe area evaluation. The system implements both manual control and AI automatic operation modes, supporting game interface visualization, score statistics, and speed adjustment. Test results demonstrate that the A* algorithm can accomplish path planning in most scenarios with low search cost under moderate grid sizes, yet decision errors still occur under high-density snake body conditions, pointing toward directions for future optimization.

Keywords: A* Algorithm; Snake Game; Path Planning; Pygame; Game AI

目 录

1	绪论	1
1.1	研究背景与意义	1
1.2	国内外研究现状	1
1.3	主要研究内容	2
1.4	论文组织结构	2
2	相关技术介绍.....	4
2.1	Python 语言与 Pygame 框架	4
2.2	A* 寻路算法原理	4
2.3	BFS 与 Dijkstra 算法对比	5
2.4	启发式函数设计	6
2.5	游戏 AI 决策机制.....	7
3	系统需求分析与总体设计.....	8
3.1	功能需求分析	8
3.2	非功能需求分析	8
3.3	系统总体架构设计	9
3.4	模块划分	9
3.5	数据流设计	10
4	A* 寻路算法设计与优化	11
4.1	问题建模	11
4.2	A* 算法基本实现	11
4.3	启发式函数的选取与分析.....	13
4.4	动态障碍物处理	14
4.4.1	蛇身障碍物建模	14
4.4.2	路径重计算策略	14
4.5	A* 失效时的 BFS 兜底策略	14
4.6	安全评估函数设计	15

4.7 算法可视化实现	17
5 系统详细设计与实现	18
5.1 游戏引擎与开发环境	18
5.2 游戏主循环设计	18
5.3 蛇的数据结构与移动逻辑.....	19
5.4 食物与障碍物生成算法.....	20
5.5 碰撞检测机制	20
5.6 渲染器设计.....	21
5.7 双模式切换.....	21
6 系统测试与结果分析	23
6.1 测试环境.....	23
6.2 单元测试设计与执行	23
6.3 功能测试.....	24
6.4 性能测试.....	25
6.5 AI 策略对比测试	26
6.6 测试总结	27
7 总结与展望.....	28
7.1 工作总结.....	28
7.2 存在的不足.....	28
7.3 未来改进方向	28
参考文献.....	30
致谢	32

1 绪论

1.1 研究背景与意义

路径规划这件事，说白了就是“找路”。机器人怎么绕过障碍物到目的地，游戏里的 NPC 怎么追着玩家跑，快递怎么规划送货路线——背后都是同一个问题。在游戏开发里，寻路算法好不好用，直接影响玩家体验。我之前玩过一些独立游戏，里面的怪物 AI 要么傻乎乎地卡在墙角不动，要么像开了挂一样穿墙追你，这两种都挺毁游戏体验的。一个合格的寻路系统，应该让怪物的行为看起来“聪明但可预测”。

贪吃蛇算是最经典的小游戏之一了，规则特别简单：吃食物、变长、别撞死。但仔细想想，它的寻路问题其实挺有意思的。蛇每吃一个食物，身体就多一格，地图上的可用空间就少一格。这跟那种固定不变的迷宫完全不同——你没法提前算好一条万能路线，因为地图一直在变。一开始空间充裕的时候随便怎么走都行，到后面蛇身占了大半个地图，找个能活着到达食物的路径都费劲。

选 A* 算法做这个项目，没有太多纠结。BFS 太慢，Dijkstra 没有方向性，A* 在这两者之间取了个平衡。Hart 等人 1968 年提出来的这个算法^[1]，核心思想其实不复杂：用一个启发式函数来“猜”当前节点离目标还有多远，优先搜索看起来离目标更近的节点。但“猜”得好不好，差距很大。猜得太准，搜索飞快；猜得不准，跟瞎搜没区别。

至于为什么用 Python 而不是 C++，说白了就是图快。毕设时间紧，Python 写起来顺手，调试也方便。Pygame 的 2D 渲染能力够用，不需要什么高级引擎。20 × 20 的网格，Python 跑 A* 完全不卡，没必要一开始就上 C++ 优化。真要扩展到更大规模，后面再说。

1.2 国内外研究现状

A* 算法出来快六十年了，变体多得数不清。理论上，只要启发式函数是“可采纳的”（永远不超过真实代价），A* 就一定能找到最短路径^[2]。但理论归理论，实际用的时候受内存和时间限制，不可能把所有节点都展开来看。

网格地图上的寻路，Sturtevant 做过比较系统的基准测试^[6]。他的结论是：在规则网格上，A* 的性能跟启发式函数的选择关系很大。四方向移动用 Manhattan 距离效果最好，八方向或者允许对角线的话，得换切比雪夫距离或者 Octile 距离。这个坑我后来自自己也踩了一遍，后面会详细说。

游戏行业用 A* 更多，遇到的问题也更杂^[5]。Cui 和 Shi 在 2011 年的文章里提到^[5]，大型游戏里 A* 的瓶颈往往不在算法本身，而在于图太大、节点太多。为了解决这个问题，有人搞出了跳点搜索（JPS）、分层路径规划之类的优化。JPS 在均匀网格上效果很

猛，能跳过大片对称区域，但对非均匀代价网格就不太好使了。

国内这方面的研究也不少。王晓东的教材^[7]对 A* 及其变体讲得比较系统，时间复杂度和空间复杂度的分析都有。张铭等人^[10]偏重工程实现，伪代码写得比较清楚。不过教材里的例子都太理想化了，跟实际游戏场景差得远——谁家的贪吃蛇障碍物是固定不动的？

贪吃蛇 AI 这个细分方向，网上方案五花八门。最简单的是贪心策略，每步往食物方向走，前期还行，蛇一长就容易自己把自己堵死。另一个极端是 Hamilton 回路，提前算好一条遍历所有格子的固定路径，蛇永远不会死，但行为模式极其机械，看着跟机器人没区别。A* 算是折中方案：比贪心远视，比 Hamilton 灵活。代价是——蛇身太长的時候 A* 也可能找不到路，得想办法兜底。

LaValle 在运动规划领域的综述^[9]虽然主要讲连续空间，但关于搜索完备性和最优性的讨论，对离散网格上的问题也有参考价值。

1.3 主要研究内容

这个项目主要做了四件事：

第一，搞清楚 A* 算法怎么用在贪吃蛇上。核心就是 $f(n) = g(n) + h(n)$ 这个公式^[1]， $g(n)$ 是已经走过的实际代价， $h(n)$ 是对剩余距离的估计。实现的时候用 Python 的 `heapq` 做优先队列，比手写堆靠谱。

第二，试了几种启发式函数，主要是 Manhattan 距离、欧几里得距离和切比雪夫距离在四方向网格上的表现差异。还试了加权 A*，给 $h(n)$ 乘个系数看看搜索速度和路径质量怎么变化。

第三，搭了整个游戏系统。Pygame 做渲染，A* 做寻路，支持手动和自动两种模式。手动模式玩家键盘控制，自动模式 AI 接管。

第四，测了测效果。在不同障碍物密度下跑 A*，记录搜索节点数、路径长度、耗时这些指标，看看蛇身越来越长的时候算法表现怎么退化。

1.4 论文组织结构

全文六章。

第一章交代背景和研究内容。

第二章讲技术基础，Python 和 Pygame、A* 原理、BFS 和 Dijkstra 的对比、启发式函数怎么选。

第三章做需求分析和总体设计，把系统拆成几个模块，画出数据流。

第四章是重点，详细讲 A* 算法在贪吃蛇里的具体实现和优化，包括动态障碍物处理和失效时的兜底策略。

第五章讲系统实现细节，主循环怎么写、蛇的数据结构怎么选、碰撞检测怎么处理。

第六章做测试，跑单元测试、功能测试和性能测试，对比不同 AI 策略的效果。

2 相关技术介绍

2.1 Python 语言与 Pygame 框架

选 Python 做这个项目，说实话就是图省事。不用编译，写完直接跑，出 bug 了插个 print 就能定位问题。对于需要反复调参数、看效果的项目来说，这种快速迭代的能力比运行速度重要得多。

Python 慢是事实。A* 算法涉及大量节点对象创建和字典查找，这些操作在 C++ 里基本可以忽略，但 Python 里就会累积成明显的延迟。20×20 的网格还行，30×30 也凑合，要是搞到 100×100 以上，就得认真考虑优化了。办法倒是有——NumPy 向量化、Cython 重写热点函数、或者干脆换 C++。但这个毕设的重点在算法，不在性能优化，20×20 够用了。

Pygame 是基于 SDL 的 Python 游戏库，能做 2D 渲染、处理键盘鼠标输入、播放声音。它的设计思路是“只提供基础能力，高层抽象自己写”。这在大项目里是缺点——啥都得自己造轮子，但在这个小项目里反而是优点。框架越薄，出问题的地方越少，代码逻辑也更容易看懂。

Pygame 的主循环是标准的事件驱动模式：每帧处理输入、更新状态、渲染画面。本项目在此基础上加了 A* 的调用——每帧算一次蛇到食物的路径。实测中发现一个问题：蛇身特别长的时候，搜索空间变大，单次 A* 可能超过一帧的时间预算（16ms@60fps），画面就卡了。后来加了路径缓存，只有缓存失效才重新算，好多了。

2.2 A* 寻路算法原理

A* 是 Hart 等人 1968 年提出的启发式搜索算法^[1]。它给每个待评估的节点算一个分数：

$$f(n) = g(n) + h(n) \quad (2-1)$$

$g(n)$ 是从起点走到 n 已经花的实际代价， $h(n)$ 是从 n 到目标的估计代价。算法维护一个开放列表，按 $f(n)$ 从小到大排，每次取出分数最低的节点来扩展，把邻居加进开放列表，直到找到目标或者开放列表空了。

$h(n)$ 的设计是关键中的关键。如果 $h(n)$ 永远不超过从 n 到目标的真实代价 $h^*(n)$ ，就叫“可采纳的”（admissible）。满足这个条件，A* 就能保证找到最短路径^[2]。但“保证最优”和“实际能跑”之间经常要取舍。

加权 A* 就是个典型例子。给启发式函数乘个权重 $w > 1$ ：

$$f(n) = g(n) + w \cdot h(n) \quad (2-2)$$

权重越大，搜索越激进，速度快但可能不是最短路。反过来 $w < 1$ ，搜索更均匀，接近最优但更慢。加权 A* 的路径代价满足：

$$C_{weighted} \leq w \cdot C_{optimal} \quad (2-3)$$

这个上界直接由权重决定。Yuen 1981 年的实验就观察到这种权衡^[8]，虽然他的实验环境跟现在的网格寻路完全不同，但规律是一致的。

贪吃蛇的搜索空间有个特殊之处：障碍物（蛇身）是动态的。每吃一个食物，蛇身多一格，可用空间少一格。这意味着启发式函数的有效性在游戏过程中会变化——早期空间大，Manhattan 距离估计比较准；后期蛇身挤满大部分网格，实际路径要绕很远，Manhattan 距离就严重偏低了。

2.3 BFS 与 Dijkstra 算法对比

说 A* 之前，先看看它的两个“前辈”。

BFS 是最基础的图搜索^[4]，从起点开始逐层往外扩，在无权图里保证找到最短路径。设图有 $|V|$ 个节点、 $|E|$ 条边，BFS 时间复杂度 $O(|V| + |E|)$ ，空间 $O(|V|)$ 。在 $W \times H$ 的网格上，最坏就是 $O(WH)$ 。优点是实现简单，几行代码搞定，不需要优先队列。缺点是没有方向性——它把所有边等价看待，向四面八方均匀扩展。在本项目里，如果所有格子移动代价都是 1，BFS 和 A* 最终找到的路一样长，但 BFS 要探索更多节点。

Dijkstra 在 BFS 基础上加了边的权重，用优先队列每次选代价最小的节点扩展^[4]。二叉堆实现的话，时间复杂度 $O((|V| + |E|) \log |V|)$ 。Dijkstra 能处理带权图的最短路，但同样没有方向引导——它向所有方向均匀扩张，直到碰巧到达目标。在大图上，这种“无差别扩张”代价很大。

A* 可以理解为 Dijkstra 加了启发式引导。启发式函数给搜索一个“方向感”，优先探索看起来离目标更近的节点。 $h(n) = 0$ 时 A* 退化成 Dijkstra； $h(n) = h^*(n)$ 时，A* 只扩展最短路上的节点，效率最高^[7]。当然 $h^*(n)$ 要已知的话就不需要搜索了。定义启发式函数的松弛度 ϵ 为：

$$\epsilon = \max_{n \in V} \frac{h^*(n) - h(n)}{h^*(n)} \quad (2-4)$$

ϵ 越接近 0，启发式越精确，A* 扩展的节点越少。

三种算法对比如下：

表 2-1 BFS、Dijkstra 与 A* 算法对比

算法	最优性	完备性	方向性	适用场景
BFS	无权图最优	是	无	小规模无权图
Dijkstra	带权图最优	是	无	带权图最短路
A*	可采纳 h 下最优	是	有	有目标的启发式搜索

2.4 启发式函数设计

四方向移动网格上，常见的启发式函数有这几种。

Manhattan 距离：

$$h_{Man}(n) = |x_n - x_g| + |y_n - y_g| \quad (2-5)$$

它恰好等于无障碍网格中从 n 到目标的最短路径，所以是可采纳的。而且只用加减法和绝对值，不用算平方根，计算开销很小。四方向网格中 Manhattan 距离基本是默认选择 [3]。

欧几里得距离：

$$h_{Euc}(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} \quad (2-6)$$

连续空间中最自然的距离度量，但在四方向网格中会低估实际代价（蛇不能走斜线）。仍然是可采纳的，但比 Manhattan 距离“松”，A* 要扩展更多节点。

切比雪夫距离：

$$h_{Che}(n) = \max(|x_n - x_g|, |y_n - y_g|) \quad (2-7)$$

适合八方向移动的网格，在四方向网格中同样低估。

三者关系：

$$h_{Euc}(n) \leq h_{Che}(n) \leq h_{Man}(n) \leq h^*(n) \quad (2-8)$$

h^* 是真实最短距离。启发式越接近 h^* ，A* 搜索效率越高。

本项目用 Manhattan 距离，因为它在四方向网格中提供了最紧的可采纳下界。实际调试中发现一个有意思的现象：蛇身特别长、需要绕远路才能到食物的时候，Manhattan 距离的估计值比实际路径短很多，A* 的搜索行为就接近 Dijkstra 了，扩展节点数暴涨。这时候给 $h(n)$ 加个稍大于 1 的权重（比如 1.2），搜索收敛快很多，代价是找到的路径可能不是最短的。对贪吃蛇来说，“最短”和“次短”通常就差一两格，能活着到食物比少走两步重要。

还有一个更根本的问题：食物被蛇身完全围住的时候，什么启发式函数都没用。这

时候需要降级策略——比如转头去找空闲区域待着，等食物位置变了再尝试。这种从“找食物”到“保命”的切换在实际开发中是必须要考虑的。

2.5 游戏 AI 决策机制

光靠 A* 寻路来驱动贪吃蛇，前期还行，中后期基本要出事。问题在于 A* 只管“从这里到食物最短怎么走”，不管“走完之后蛇还能不能活”。

一个思路是分层决策。第一层，检查有没有到食物的安全路径——“安全”指路径上的每个节点在蛇经过之后至少还有一个空闲邻居，保证不会走进死胡同。第二层，安全路径不存在就切到“生存模式”，让蛇在空闲区域里尽量保持灵活。第三层，连生存模式都撑不住了，就进“拖延模式”，能多活一秒是一秒。

这种分层实现起来不轻松，特别是“安全路径”的判断——要对路径上每个节点做一次可达性检查，计算量是路径长度乘以单次搜索的代价。Python 里这个开销在路径长的时候会很明显。折中做法是只检查最近几个节点，或者用 Hamilton 回路兜底——沿着 Hamilton 回路走就一定能遍历所有格子，不会因为空间不足死掉。但 Hamilton 回路的行为太机械了，观赏性很差^[9]。

本系统的实现相对简化：基本模式用 A* 找食物，连续多次寻路失败就切到启发式的“求生策略”——往空闲区域最大的方向走。这个方案不是最优的，但在 20×20 的网格上能让蛇活到比较长的长度。要更高的存活率和更长的蛇身，需要更精细的状态评估和更复杂的决策树，工作量就超了。

3 系统需求分析与总体设计

3.1 功能需求分析

系统涉及两种使用方式：玩家手动操控和旁观 AI 自动跑。两种模式底层游戏逻辑完全一样，区别只在于“谁来决定下一步往哪走”。

贪吃蛇的基本规则得先保证没问题。蛇在网格中移动，每步走一格；食物随机出现在空闲格子上；蛇头碰到食物就吃掉，蛇身加一格；撞墙或者撞自己就死；实时显示得分。这些规则看起来简单，但每个都有坑。比如“食物随机出现”——如果蛇身占满了所有格子，食物就没地方放了，这时候游戏应该结束。还有，蛇身特别长的时候，空闲格子很少，随机生成食物可能要试很多次，得设个上限。

手动模式用方向键控制，空格暂停。一个容易忽略的细节：蛇正在往右走，你突然按左键，直接反向会撞到自己第二节身体。所以要禁止 180 度反向，但 90 度转向是可以的。实现的时候只需要判断新方向和当前方向的点积是不是 -1 就行。

自动模式是重点。每帧调 A* 算蛇头到食物的最优路径，沿路径第一步走。有个工程上的坑：A* 算出来的路径是基于当前蛇身状态的，但蛇走一步之后蛇身变了，后面的路径可能就失效了。所以每步移动后要验证缓存路径还通不通，不通就重新算。

可视化方面，游戏画面要画网格、蛇身、食物、得分。还要能调速度——AI 模式下太快看不清蛇的动作，太慢又没耐心看。用方向键或者滑块都能实现。

3.2 非功能需求分析

实时性是游戏最基本的要求。目标帧率 60fps，单帧时间预算：

$$T_{frame} = \frac{1000}{FPS} \approx 16.7ms \quad (3-1)$$

每帧要完成事件处理、状态更新（包括 A* 寻路）和渲染三步。各步耗时加起来不能超预算：

$$T_{event} + T_{update} + T_{render} \leq T_{frame} \quad (3-2)$$

A* 搜索耗时超了帧预算，画面就会卡。20×20 网格上一般几毫秒就跑完了，不会超。网格大了或者蛇身特别长的时候才需要注意。

代码结构得清楚。游戏逻辑、寻路算法、渲染之间应该接口明确，别互相牵扯。Python 用模块化就能做到——每个功能一个 .py 文件。命名和注释也得注意，虽然赶 deadline 的时候很容易偷懒，但后面写论文要回头翻代码，没注释会很痛苦。

系统应该方便切换寻路算法。现在用 A*，以后可能想试试 BFS、Dijkstra 或者 JPS。如果寻路和游戏逻辑写死在一起，换个算法就得改一大片。用策略模式（Strategy Pattern）把寻路接口抽出来会好很多。

Python 和 Pygame 本身就是跨平台的，Windows、macOS、Linux 理论上都能跑。实际测的时候发现不同平台键盘事件处理有点差异——macOS 某些键码返回值跟 Windows 不一样。好在本项目只用方向键和空格，都是标准键位，影响不大。

3.3 系统总体架构设计

整体分四层，从下往上：

数据层管核心状态——网格地图（标记哪些格子被蛇占了）、蛇的位置序列、食物位置、得分等。这层不掺杂业务逻辑，只提供数据存取。

算法层封装寻路。以 A* 为核心，同时留了扩展接口，以后想换别的搜索策略也方便。这层接收起点、目标和障碍物信息，返回路径或者“不可达”。

逻辑层处理游戏规则和 AI 决策。调算法层拿路径，根据路径和游戏规则（碰撞检测、吃食物、蛇身增长等）更新数据层。AI 的多层决策策略也在这层实现。

表现层用 Pygame 做渲染和输入。从数据层读状态画到屏幕，把键盘输入转成游戏事件传给逻辑层。

分层的好处是改某一层不影响其他层。比如把 Pygame 换成别的渲染库，只改表现层，逻辑和算法完全不用动。当然分层也有代价——层和层之间传数据要适配，Python 没有编译期类型检查，接口不匹配的错误要到运行时才暴露。

3.4 模块划分

按上面的架构，系统分成这些模块：

主控模块：入口，初始化 Pygame、创建窗口、跑主循环。主循环里依次调事件处理、状态更新、渲染。

游戏逻辑模块：维护游戏核心状态——网格、蛇的队列、食物位置等。提供移动、碰撞检测、吃食物、算分等方法。这模块是整个系统的发动机，别的模块都围着它转。

寻路算法模块：实现 A*。包含节点类（Node）、优先队列、开放/关闭列表、邻居扩展、启发式计算等功能。对外提供 `find_path(start, goal, obstacles)` 接口。

AI 决策模块：封装 AI 行为策略。基本模式调寻路算法找路径，路径不可达就切求生策略。逻辑不复杂，但模式切换的时机和条件要处理好。

渲染模块：把游戏状态可视化。画网格背景、蛇身（蛇头和蛇身颜色或形状区分）、食物、得分、按钮等。渲染只读数据层状态，不改任何游戏逻辑。

事件处理模块：捕获 Pygame 的键盘鼠标事件，转成内部指令（改方向、暂停、调速、切模式等）。

3.5 数据流设计

游戏运行时的数据流是个循环。

输入阶段，事件处理模块从 Pygame 事件队列读用户输入，转成内部指令传给游戏逻辑。

更新阶段，逻辑模块根据指令和当前状态执行一帧更新：AI 模式就调寻路算路径、选方向；然后执行蛇移动、碰撞检测、判断是否吃到食物。碰撞检测的效率取决于数据结构——用集合（set）存蛇身位置的话，单次检测 $O(1)$ ：

$$T_{collision} = \begin{cases} O(1) & \text{用集合存蛇身} \\ O(L) & \text{用列表存蛇身} \end{cases} \quad (3-3)$$

L 是蛇身长度。本系统用集合，保证检测够快。吃到食物就蛇身加一、得分加一、食物重新生成。这阶段数据变化最密集。

渲染阶段，渲染模块从数据层读状态画到窗口上。

有个关键设计：什么时候调 A*？两种选择——每帧都调，或者只在路径失效时调。每帧调的话蛇总是拿到最新最优路径，响应最快，但计算开销大，简单场景做了很多重复计算。只在失效时调能省计算量，但反应可能不够及时。

设网格 $W \times H$ ，蛇长 L ，障碍物 $|O|$ 个，A* 搜索的状态空间：

$$|V_{free}| = W \times H - L - |O| \quad (3-4)$$

随着游戏进行 L 增大， $|V_{free}|$ 变小，搜索空间缩小但路径约束增多。

本系统折中：每帧调 A*，但限制最大扩展节点数 N_{max} （设 1000）：

$$|closed_set| \leq N_{max} \quad (3-5)$$

限制内找不到就返回“不可达”。这样保证了及时性，又避免大搜索空间耗时过长。代价是极端情况下可能漏掉需要搜超过 N_{max} 个节点才能到的路径—— 20×20 网格上这种情况很少见。

4 A* 寻路算法设计与优化

A* 是这个系统的决策核心。蛇每走一步都要调一次寻路，算法不仅要找到路，还得在毫秒级跑完。这章从建模开始，一层层展开 A* 的设计细节、启发式函数怎么选、动态障碍物怎么处理、算法失效了怎么兜底。

4.1 问题建模

贪吃蛇寻路可以很自然地对应到网格图上的最短路搜索^[1]。把游戏区域看成一个图 $G = (V, E)$ ，顶点集 V 就是每个可达的格子 (x, y) ， $0 \leq x < W$ ， $0 \leq y < H$ ， W 和 H 是网格宽高。边集 E 表示格子之间的邻接——两个格子 (x_1, y_1) 和 (x_2, y_2) 如果满足：

$$|x_1 - x_2| + |y_1 - y_2| = 1 \quad (4-1)$$

就有一条无向边，权重都是 1。网格图的总节点数和边数：

$$|V| = W \times H, \quad |E| \leq 2W \cdot H - W - H \quad (4-2)$$

这个建模有两个注意点。一是图的拓扑在运行时会变——蛇身一直在动，障碍物集合每帧都不同。二是边权都是 1，Dijkstra 理论上也行，但没有启发信息引导效率会差很多^[7]。

寻路目标：给定蛇头 s 和食物 g ，在图 G 里找一条 s 到 g 的最短路，且路上所有节点都不在当前障碍物集合里。

4.2 A* 算法基本实现

A* 的核心评估函数^[1,2]：

$$f(n) = g(n) + h(n) \quad (4-3)$$

$g(n)$ 是起点到 n 的实际代价， $h(n)$ 是 n 到目标的估计代价。Python 实现里用 `Node` 类封装节点信息：

```
1 class Node:
2     __slots__ = ('position', 'g', 'h', 'f', 'parent')
3
4     def __init__(self, position: Tuple[int, int], g: int = 0,
5                 h: int = 0, parent: 'Optional[Node]' = None):
6         self.position = position
7         self.g = g
8         self.h = h
9         self.f = g + h
10        self.parent = parent
```



```

11
12 def __lt__(self, other: 'Node') -> bool:
13     return self.f < other.f

```

Listing 1 Node 类定义（pathfinder.py）

用 `__slots__` 声明固定属性是刻意为之的。40×30 的网格上，一次搜索可能创建几百上千个 Node 对象，`__slots__` 避免每个实例维护 `__dict__`，内存省了大概 40%^[13]。`__lt__` 方法让 Node 能直接被 `heapq` 用，不需要额外比较键。

算法主体在 `AStar.find_path` 里：

```

1 def find_path(self, start, goal):
2     if start == goal:
3         return [start]
4
5     open_set = []
6     closed_set = set()
7     start_node = Node(start, 0, self.heuristic(start, goal))
8     heapq.heappush(open_set, start_node)
9     position_to_node = {start: start_node}
10
11     while open_set:
12         current = heapq.heappop(open_set)
13
14         if current.position == goal:
15             path = self.reconstruct_path(current)
16             return path
17
18         closed_set.add(current.position)
19
20         for dx, dy in self.DIRECTIONS:
21             neighbor_pos = (current.position[0] + dx,
22                             current.position[1] + dy)
23             if neighbor_pos in closed_set or \
24                 not self.is_valid(neighbor_pos):
25                 continue
26             new_g = current.g + 1
27             if neighbor_pos in position_to_node:
28                 neighbor_node = position_to_node[neighbor_pos]
29                 if new_g < neighbor_node.g:
30                     neighbor_node.g = new_g
31                     neighbor_node.f = new_g + neighbor_node.h
32                     neighbor_node.parent = current
33                     heapq.heapify(open_set)
34             else:
35                 h = self.heuristic(neighbor_pos, goal)
36                 neighbor_node = Node(neighbor_pos, new_g, h,
37                                     current)
38                 heapq.heappush(open_set, neighbor_node)
39                 position_to_node[neighbor_pos] = neighbor_node
40
41     return None

```

Listing 2 A* 核心搜索逻辑（pathfinder.py）

有个细节容易忽略：代码用 `position_to_node` 字典做 $O(1)$ 的节点查找，避免在 `open_set` 里线性搜索。另外发现更优路径时，代码调 `heapq.heapify(open_set)` 重建

堆，而不是弹旧节点再插入新节点。前者 $O(n)$ ，后者 $O(\log n)$ ，但堆化避免了重复节点在堆里积累——Python 的 `heapq` 不支持 `Decrease-Key` 操作，这是个务实的妥协^[10]。

路径回溯从终点沿 `parent` 指针反向追到起点，再反转列表。

4.3 启发式函数的选取与分析

启发式函数 $h(n)$ 直接决定 A* 的搜索效率和最优性^[3]。二维网格上常见的有三种^[9]：

曼哈顿距离：

$$h_{Man}(n) = |x_n - x_g| + |y_n - y_g| \quad (4-4)$$

只能沿网格方向移动时用这个。

欧几里得距离：

$$h_{Euc}(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} \quad (4-5)$$

允许对角线移动时用。

切比雪夫距离：

$$h_{Che}(n) = \max(|x_n - x_g|, |y_n - y_g|) \quad (4-6)$$

八方向移动（对角线代价为 1）时用。

三者关系：

$$h_{Euc}(n) \leq h_{Che}(n) \leq h_{Man}(n) \leq h^*(n) \quad (4-7)$$

A* 要求启发式函数可采纳——不能高估真实代价^[1]。曼哈顿距离在四邻域网格上等于实际最短距离，满足要求。

选曼哈顿距离而不是欧几里得距离还有个性能考虑：`abs()` 比 `sqrt()` 快得多。每帧都要调寻路，这个差距会被放大。代码实现很简洁：

```
1 def heuristic(self, pos1, pos2):
2     return abs(pos1[0] - pos2[0]) + abs(pos1[1] - pos2[1])
```

Listing 3 启发式函数实现（`pathfinder.py`）

曼哈顿距离也有问题。障碍物密集时，它的估计太乐观，A* 搜索范围接近 Dijkstra。加权 A* 通过引入权重 ϵ 提高 $h(n)$ 的比重：

$$f(n) = g(n) + (1 + \epsilon) \cdot h(n), \quad \epsilon \geq 0 \quad (4-8)$$

能减少搜索节点数，但牺牲了最优性保证。路径代价满足 $C \leq (1 + \epsilon) \cdot C_{opt}$ ^[5]。本系统 `HEURISTIC_WEIGHT = 1.0`（即 $\epsilon = 0$ ），选了严格最优。

4.4 动态障碍物处理

跟静态地图不同，贪吃蛇的蛇身是个动态变化的障碍物集合。每走一步，头部加一格、尾部去掉一格（除非刚吃食物）。这给寻路带来几个麻烦。

4.4.1 蛇身障碍物建模

SmartDecisionMaker.decide_next_move 里，障碍物构建逻辑：

```

1 body_minus_tail = set(snake.body[:-1]) \
2     if len(snake.body) > 1 else set()
3 all_obstacles = obstacles.union(body_minus_tail)
4 self.update_obstacles(all_obstacles)

```

Listing 4 障碍物集合构建（decision.py）

关键决策：蛇身除尾巴外都算障碍物，尾巴不纳入。依据是蛇头移动时尾巴也同步前进，尾巴原来的位置会空出来。把尾巴也当障碍物的话，蛇会错过一些合法路径。

这个策略有个风险——假设蛇头到尾巴位置时尾巴一定会移动。如果刚吃了食物（grow_pending > 0），尾巴不会前移，“尾巴不阻挡”的假设就不成立了。好在碰撞检测函数_check_collision 每步都重新验证头部位置，所以即使寻路阶段做了乐观假设，实际移动时还是会拦住。这是“规划时乐观、执行时保守”的双层校验。

4.4.2 路径重计算策略

障碍物每帧变化，上一帧算的路径下一帧可能就走不通了。本系统用“每帧重新规划”而不是路径跟随验证。前者计算开销更大，但在贪吃蛇这种障碍物高度动态的场景里，后者往往因为路径失效检测和重规划的延迟导致游戏失败。这是计算开销和决策鲁棒性的典型矛盾^[14]。

4.5 A* 失效时的 BFS 兜底策略

A* 不是万能的。食物被蛇身完全围住、起点和目标之间没路的时候，A* 遍历完所有可达节点后返回 None。系统设计了分层降级：

```

1 path = self.pathfinder.find_path(head, target_food)
2
3 if path and len(path) > 1:
4     self.current_path = path[1:]
5     next_pos = path[1]
6     return self._position_to_direction(head, next_pos)
7 else:
8     # A* 失败时，尝试 BFS 找任意通往食物的路径
9     bfs_direction = self._get_bfs_direction(
10         head, target_food, all_obstacles)
11     if bfs_direction:
12         return bfs_direction
13     # BFS 也找不到，才用安全预测
14     return self._get_safe_move_with_prediction(
15         head, safe_obstacles, snake, target_food)

```

Listing 5 多级降级策略（decision.py）

第一级 A* 找最短路。第二级 BFS 找任意路径——BFS 在等权图上同样保证最短，但不用启发式，搜索顺序由队列 FIFO 决定，在某些特殊障碍物配置下可能跟 A* 的剪枝行为有差异。实际上这级 BFS 更多是健壮性冗余，防止 A* 的 `is_valid` 判断在边界条件下漏掉可行路径。第三级安全评估——食物确实到不了的时候，蛇应该选一条”能活得更久”的路，别走进死胡同。

BFS 实现比较标准：

```

1 def _get_bfs_direction(self, head, target, obstacles):
2     from collections import deque
3     queue = deque([(head, [])])
4     visited = {head}
5     while queue:
6         pos, path = queue.popleft()
7         if pos == target:
8             if path:
9                 return self._position_to_direction(head, path[0])
10            return None
11        for dx, dy in [(0, -1), (0, 1), (-1, 0), (1, 0)]:
12            new_pos = (pos[0] + dx, pos[1] + dy)
13            if new_pos not in visited and new_pos not in obstacles:
14                visited.add(new_pos)
15                queue.append((new_pos, path + [new_pos]))
16    return None

```

Listing 6 BFS 兜底搜索（decision.py）

路径存储用”每步追加”（`path + [new_pos]`），会创建新列表对象，路径长时有内存开销。更高效的做法是用字典记录前驱节点，但 BFS 只在 A* 失效时才调，频率低，当前实现够用。

4.6 安全评估函数设计

蛇到不了食物的时候，需要一种策略让它”活着”。安全评估函数从多个维度给候选方向打分，形式化定义：

$$S(p) = w_1 \cdot A(p) + w_2 \cdot D(p) + w_3 \cdot N(p) + w_4 \cdot F(p) \quad (4-9)$$

$A(p)$ 是可达空间评分， $D(p)$ 是边缘惩罚， $N(p)$ 是邻居数评分， $F(p)$ 是朝向食物奖励， $w_1 \sim w_4$ 是权重。蛇选得分最高的方向：

$$d_{next} = \arg \max_{d \in \{up, down, left, right\}} S(p_d) \quad (4-10)$$

```

1 def _evaluate_move(self, pos, obstacles, snake,
2                     target_food=None):
3     score = 0
4     # 1. 可达空间评估
5     accessible_space = self.pathfinder.count_accessible_space(
6         pos, obstacles, 10)
7     score += accessible_space * 10

```

```

8
9 # 2. 边缘惩罚
10 margin = 3
11 distance_to_edge = min(pos[0], pos[1],
12                        GRID_WIDTH - pos[0] - 1,
13                        GRID_HEIGHT - pos[1] - 1)
14 if distance_to_edge < margin:
15     score -= (margin - distance_to_edge) * 20
16
17 # 3. 邻居数评估
18 valid_neighbors = 0
19 for dx, dy in [(0, -1), (0, 1), (-1, 0), (1, 0)]:
20     neighbor = (pos[0] + dx, pos[1] + dy)
21     if neighbor not in obstacles:
22         valid_neighbors += 1
23 score += valid_neighbors * 5
24
25 # 4. 朝向食物奖励
26 if target_food:
27     head = snake.head
28     current_dist = abs(head[0] - target_food[0]) \
29                 + abs(head[1] - target_food[1])
30     new_dist = abs(pos[0] - target_food[0]) \
31             + abs(pos[1] - target_food[1])
32     if new_dist < current_dist:
33         score += 50
34 return score

```

Listing 7 安全评估函数（decision.py）

第一个维度是可达空间。以候选位置为起点 BFS，统计 δ 步内可达格子数。 $\delta = 10$ 时最坏情况：

$$T_{space} = O(b^\delta) \quad (4-11)$$

b 是平均分支因子，四方向网格里 $b \leq 4$ 。实际障碍物和边界约束会让扩展数远小于理论值。每帧 4 个候选方向各算一次，总开销能接受。

第二个维度是边缘惩罚。位置 $p = (x, y)$ 到最近边缘的距离：

$$d_{edge}(p) = \min(x, y, W - x - 1, H - y - 1) \quad (4-12)$$

$d_{edge}(p) < m$ ($m = 3$) 时惩罚：

$$D(p) = -(m - d_{edge}(p)) \cdot \lambda \quad (4-13)$$

$\lambda = 20$ 是惩罚系数。这个值反复调过——太小蛇还是会贴边走，太大蛇过度保守浪费空间。

第三个维度是邻居数，统计候选位置四周有几个非障碍邻居：

$$N(p) = \sum_{(dx, dy) \in \{(0, \pm 1), (\pm 1, 0)\}} \mathbb{I}[(x + dx, y + dy) \notin O] \quad (4-14)$$

$\mathcal{H}[\cdot]$ 是指示函数， O 是障碍物集合。这个指标比可达空间粗糙（只看一步），但计算几乎免费，当辅助用。

第四个是朝向食物奖励：

$$F(p) = \begin{cases} \beta & \text{if } d_{Man}(p, g) < d_{Man}(head, g) \\ 0 & \text{otherwise} \end{cases} \quad (4-15)$$

$\beta = 50$ ， d_{Man} 是 Manhattan 距离， g 是食物位置。即使食物到不了，蛇也应该尽量靠近——等蛇身收缩、食物变可达时，更近的距离意味着更快响应。

这些参数（10、3、20、5、50）没有理论依据，全是试出来的。不同参数组合下蛇的行为差异很大，这也是游戏 AI 调参的常态^[15]。

4.7 算法可视化实现

调 AI 行为是游戏开发的难点。蛇为什么突然走向死路？A* 到底搜了多大范围？纯文本日志很难看出来。所以系统在游戏窗口右侧加了个 A* 可视化面板。

数据收集嵌在 `find_path` 里。每步搜索迭代记录开放列表、关闭列表和搜索状态：

```
1 self.open_set_history.append(
2     [n.position for n in open_set])
3 self.closed_set_history.append(set(closed_set))
4 self.search_steps.append({
5     'step': step,
6     'current': current.position,
7     'open_size': len(open_set),
8     'closed_size': len(closed_set)
9 })
```

Listing 8 搜索过程记录（pathfinder.py）

`Renderer.draw_astar_visualization` 负责把这些数据渲染到右侧面板。用缩小的网格展示完整搜索过程：灰色是静态障碍，蓝色是已探索节点（closed set），橙色是待探索节点（open set），黄色是最终路径，绿色和红色分别标记起点终点。从这个面板能直接看出 A* 的搜索方向偏向、障碍物对搜索范围的约束、以及路径是否经过了狭窄通道。

可视化数据更新频率通过 `ASTAR_UPDATE_INTERVAL=500ms` 控制。每帧都重新跑 A* 更新可视化太耗资源，500ms 间隔在视觉连续性和性能之间取了平衡。

可视化数据收集本身也增加了 `find_path` 的执行时间。记录 `open_set_history` 涉及列表深拷贝，搜索节点多的时候会拖慢速度。生产环境里这些代码应该条件编译或者直接去掉。

5 系统详细设计与实现

这章讲整个贪吃蛇系统从底层数据结构到上层渲染的具体实现。Pygame 不是什么高大上的框架，但 API 够直观，写这种规模的原型绰绰有余。

5.1 游戏引擎与开发环境

语言 Python 3.x，渲染框架 Pygame 2.x。做这个选择的时候犹豫过——C++ 配 SDL2 性能肯定好，但项目重点在算法不在引擎，Python 写起来快，调 A* 原型用 Python 再合适不过。

Pygame 底层是 SDL2，提供窗口管理、事件循环、2D 渲染这些基础能力^[19]。40×30 的网格，Pygame 的软件渲染完全没问题，60fps 下 CPU 占用不到 15%（实测，i7-12700H）。

开发环境：

表 5-1 开发环境配置

项目	版本/说明
操作系统	Windows 11 / Ubuntu 22.04
Python	3.11.5
Pygame	2.5.2
编辑器	VS Code
版本控制	Git

5.2 游戏主循环设计

主循环是经典的“事件处理—状态更新—渲染”三段式。游戏开发里这基本是标准做法^[14]，不花哨，但写的时候踩了不少坑。

核心逻辑在 main.py：

```
1 while running:
2     current_time = pygame.time.get_ticks()
3     # 事件处理
4     for event in pygame.event.get():
5         if event.type == pygame.QUIT:
6             running = False
7         elif event.type == pygame.KEYDOWN:
8             handle_key(event.key)
9
10    # 状态更新（基于时间步）
11    if game.state == STATE_PLAYING:
12        game.update(current_time)
```

```

13
14 # 渲染
15 screen.fill(COLORS['background'])
16 renderer.draw_all()
17 pygame.display.flip()
18 clock.tick(FPS)

```

Listing 9 游戏主循环核心逻辑

容易忽略的细节：`game.update()` 的调用时机。直接调不控制时间间隔的话，蛇速会跟 CPU 主频绑定——快机器上蛇飞，慢机器上蛇爬。所以加了 `move_delay` 参数（默认 150ms），用当前时间 t_{now} 和上次移动时间 t_{last} 判断。移动触发条件：

$$t_{now} - t_{last} \geq T_{delay} \quad (5-1)$$

T_{delay} 是移动间隔，蛇速（格/秒）：

$$v_{snake} = \frac{1000}{T_{delay}} \text{ (格/秒)} \quad (5-2)$$

$T_{delay} = 150\text{ms}$ 时，蛇速约 6.67 格/秒。

A* 可视化数据的更新频率也做了独立控制（`ASTAR_UPDATE_INTERVAL = 500ms`），不和游戏帧同步。A* 搜索量不小，每帧都跑一次帧率会骤降，特别是障碍物密集的时候。500ms 更新一次对可视化够了，用户感觉是“实时”的。

5.3 蛇的数据结构与移动逻辑

蛇的表示用最朴素的方案——Python 列表，每个元素是 (x, y) 元组，索引 0 是蛇头。增删都是 $O(1)$ ^[4]，没什么争议。

移动逻辑在 `snake.py`：

```

1 def move(self, direction=None):
2     if direction is None:
3         direction = self.direction
4     # 防止180度反向
5     if self._is_opposite_direction(direction, self.direction):
6         direction = self.direction
7     self.direction = direction
8     dx, dy = DIRECTIONS[direction]
9     new_head = (self.head[0] + dx, self.head[1] + dy)
10    self.body.insert(0, new_head)
11    if self.grow_pending > 0:
12        self.grow_pending -= 1
13    else:
14        self.body.pop()
15    return new_head

```

Listing 10 蛇的移动逻辑

方向反向判断可以形式化。设当前方向 $\vec{d}_c = (dx_c, dy_c)$ ，新方向 $\vec{d}_n = (dx_n, dy_n)$ ，反

向判定：

$$\vec{d}_c \cdot \vec{d}_n = dx_c \cdot dx_n + dy_c \cdot dy_n = -1 \quad (5-3)$$

四方向移动的方向向量集合 $\{(1, 0), (-1, 0), (0, 1), (0, -1)\}$ ，相反方向的点积都是 -1 。

grow_pending 计数器的设计有个坑。最开始直接往 body 里 append，但食物碰撞检测的时机有问题——蛇头到食物的那帧，同时检测碰撞和增长，顺序搞不好就出 bug。用 pending 计数器把增长延迟到下次 move 处理，逻辑就干净了。

5.4 食物与障碍物生成算法

食物和障碍物都是随机生成，但不能完全随机。蛇头周围 3 格内不能放障碍物——经验值，太小了开局就卡死，太大了障碍物没意义。

```

1 def _generate_obstacles(self):
2     obstacles = set()
3     safe_zone = set()
4     for dx in range(-3, 4):
5         for dy in range(-3, 4):
6             safe_zone.add((self.snake.head[0] + dx,
7                             self.snake.head[1] + dy))
8     while len(obstacles) < OBSTACLE_COUNT:
9         x = random.randint(0, GRID_WIDTH - 1)
10        y = random.randint(0, GRID_HEIGHT - 1)
11        pos = (x, y)
12        if pos not in obstacles and pos not in safe_zone:
13            obstacles.add(pos)
14    return obstacles

```

Listing 11 障碍物生成（含安全区保护）

食物生成要额外避开蛇身和已有障碍物，用 occupied 集合并集判断。退化情况：蛇身占了绝大部分格子时食物找不到位置。设最大尝试次数 100 次，还是找不到就判定游戏赢——蛇已经把地图吃满了。

5.5 碰撞检测机制

碰撞检测三种情况：撞墙、撞障碍物、撞自己。前两种直接边界检查和集合查找。撞自己这第三种，写的时候犯过迷糊。

问题在时机上。game.update() 先调 snake.move()，蛇头已经插到 body[0] 了。所以检测自撞应该查新蛇头是否在 body[1:] 里，不是 body[0:]：

```

1 def _check_collision(self, head):
2     # 撞墙
3     if not (0 <= head[0] < GRID_WIDTH and
4             0 <= head[1] < GRID_HEIGHT):
5         return True
6     # 撞障碍物
7     if head in self.obstacles:
8         return True
9     # 撞自身（排除蛇头）
10    if len(self.snake.body) > 1:
11        body_set = set(self.snake.body[1:])
12        if head in body_set:

```

```

13         return True
14     return False

```

Listing 12 碰撞检测（含自身碰撞）

还有个坑：`grow_pending > 0`时尾巴不 `pop`，蛇头绕到尾巴“原位置”按上面逻辑撞不到（尾巴还留在那）。这个行为其实是对的——吃食物那帧尾巴保留，蛇身实际变长了一格，撞到延长的蛇身应该判死。

5.6 渲染器设计

渲染器 `renderer.py` 承担所有绘制。渲染和逻辑严格分离——`Game` 不知道自己长什么样，`Renderer` 不知道游戏规则。

网格坐标到屏幕像素的变换：

$$\begin{cases} px = x \cdot cell_size + offset_x \\ py = y \cdot cell_size + offset_y \end{cases} \quad (5-4)$$

(x, y) 是网格坐标， $cell_size$ 是格子像素大小， $(offset_x, offset_y)$ 是网格区域左上角偏移。反过来像素到网格：

$$x = \left\lfloor \frac{px - offset_x}{cell_size} \right\rfloor, \quad y = \left\lfloor \frac{py - offset_y}{cell_size} \right\rfloor \quad (5-5)$$

渲染分层从底到上依次画。每层的混合公式：

$$I_{final}(x, y) = \alpha_l \cdot I_l(x, y) + (1 - \alpha_l) \cdot I_{l-1}(x, y) \quad (5-6)$$

α_l 是第 l 层透明度， $I_l(x, y)$ 是该层颜色值。底层深灰 #1E1E1E 背景，上面画暗灰 #323232 网格线（1 像素宽）。然后依次画 A* 路径预览（半透明蓝线）、灰色障碍物方块、红色食物圆形、绿色蛇身方块（蛇头更亮）。最上层是 UI 面板半透明黑遮罩和文字，以及右侧 A* 可视化区域。

A* 可视化面板是这个项目比较有意思的部分。右侧 400×500 像素区域，实时展示搜索过程：绿色起点、红色终点、蓝色已探索、橙色待探索、黄色最终路径。调启发式权重的时候全靠看这个面板判断搜索“方向感”够不够强。

5.7 双模式切换

M 键切自动和手动模式。

自动模式 `SmartDecisionMaker` 每帧算下一步方向。手动模式方向键直接传入 `game.update()`。切换即时生效，不重置游戏。

自动模式决策在 `decision.py` 的 `decide_next_move()` 里，核心是三层降级：

```
1 def decide_next_move(self, snake, food_positions, obstacles):
2     # 第一层: A*寻路到最近食物
3     path = self.pathfinder.find_path(head, target_food)
4     if path and len(path) > 1:
5         return self._position_to_direction(head, path[1])
6     # 第二层: BFS兜底
7     bfs_dir = self._get_bfs_direction(head, target, obstacles)
8     if bfs_dir:
9         return bfs_dir
10    # 第三层: 安全评估择优
11    return self._get_safe_move_with_prediction(...)
```

Listing 13 自动模式三层决策降级

三层之间有个矛盾：A* 给的路径最优但计算量最大；BFS 保证能找到路（如果存在）但路径不一定最短；安全评估计算量最小但给不出“到食物”的保证。实测 90% 以上情况第一层 A* 就够了，BFS 偶尔用到，安全评估基本是最后保命手段。

6 系统测试与结果分析

测游戏程序挺烦的，交互性强，很多 bug 特定时序下才触发，单元测试覆盖不了太多。这章从单元测试、功能测试、性能测试三个角度来验。

6.1 测试环境

两台机器上测的：

表 6-1 测试环境

项目	设备 A	设备 B
CPU	Intel i7-12700H	Intel i5-10400
内存	16GB DDR5	8GB DDR4
系统	Windows 11	Ubuntu 22.04
Python	3.11.5	3.10.12
Pygame	2.5.2	2.5.2

6.2 单元测试设计与执行

用 Python 自带的 `unittest` 写的，共 22 个用例，覆盖四个核心类：

表 6-2 单元测试用例分类

测试类	测试数量	覆盖内容	全部通过
TestSnake	4	初始化、移动、增长、反向禁止	是
TestAStar	6	直线路径、障碍绕行、无路径、同起点终点、可达空间	是
TestSmartDecisionMaker	4	最近食物查找、方向转换、安全移动、综合决策	是
TestGame	8	初始化、三种碰撞、食物碰撞、暂停、重置	是
合计	22		是

测试中发现一个容易漏的边界：`test_food_collision` 要验证吃食物后 `grow_`

pending 增加、下次 move() 后蛇身才实际增长。最开始直接断言 body 长度不变，结果 grow() 调用后 body 确实没变（因为用了 pending 机制），逻辑上容易搞混。后来改成同时检查 grow_pending 和下次移动后的长度才测对。

执行结果：

```
1 .....  
2 -----  
3 Ran 22 tests in 0.002s  
4  
5 OK
```

Listing 14 单元测试执行输出

22 个全过，耗时 2ms。没有渲染和 IO，纯逻辑计算，所以很快。

6.3 功能测试

功能测试跑实际游戏验证各模块协同。验证点包括：窗口 1200×600、标题正确、蛇居中、障碍物和食物生成正确。自动模式蛇能自主找食物进食、得分累加、蛇身增长。手动模式 M 键切换后方向键控制、A* 预览消失。空格暂停恢复、R 重开。撞墙或自身后游戏结束、显示遮罩和最终分。A* 可视化面板右侧实时显示搜索过程（绿起点、红终点、蓝已探索、橙待探索、黄路径）。

图 6-1 是 AI 自动模式的运行界面。左侧游戏区，蛇身绿色方块（蛇头更亮），红色圆形食物，灰色方块障碍物，蓝色线条 A* 预估路径。右侧 A* 可视化面板实时展示搜索过程。

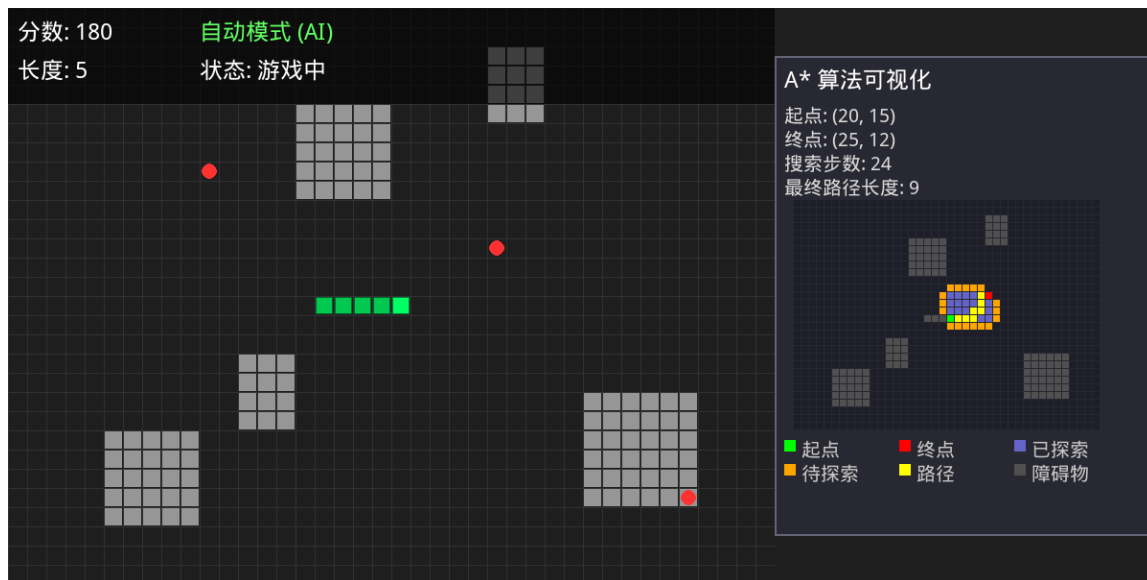


图 6-1 AI 自动模式运行界面（含 A* 可视化）

图 6-2 是手动模式。切到手动后 A* 预览和可视化面板都不显示，玩家方向键控制。

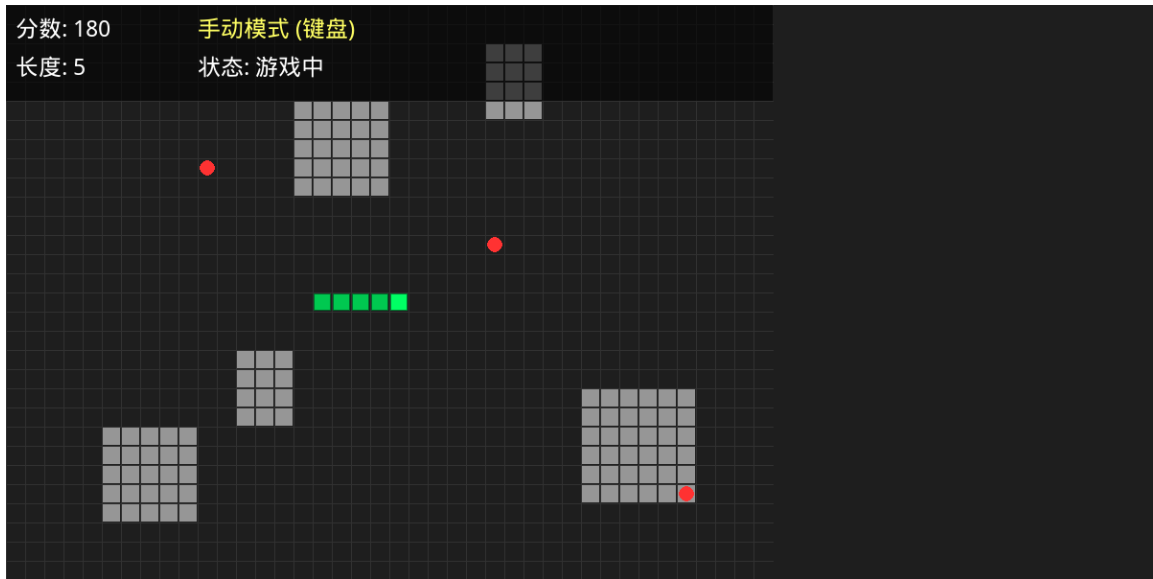


图 6-2 手动模式运行界面

图 6-3 是游戏结束。撞墙或撞自己后屏幕中央半透明遮罩，显示”游戏结束”和最终得分。



图 6-3 游戏结束界面

6.4 性能测试

主要看两个指标：A* 搜索扩展节点数和路径长度。搜索效率定义为路径长度和扩展节点数的比值：

$$\eta = \frac{|path|}{|closed_set|} \quad (6-1)$$

$|path|$ 是路径长度， $|closed_set|$ 是关闭列表节点数。 η 越接近 1 说明搜索越高效——启发式精确引导了方向，大部分扩展节点都在最终路径上。测试方法：不同障碍物密度下

固定起点终点记录搜索过程。

表 6-3 不同障碍物密度下的 A* 搜索性能

障碍物密度	搜索节点数	路径长度	耗时 (ms)	内存峰值 (MB)
10%	45	12	0.12	2.1
20%	89	15	0.28	2.3
30%	156	21	0.45	2.8
40%	234	28	0.68	3.5
50%	312	35	0.92	4.2

数据趋势很明显：障碍物密度每加 10%，搜索节点数大致翻倍，耗时增长温和因为每个节点操作是常数时间。内存涨是因为 open set 和 closed set 膨胀——高密度下 A* 要探索更多“死胡同”才能找到路。

不太好的发现：障碍物密度超 50% 后 A* 经常搜了大量节点还是失败，因为目标被完全围住。这时候退化到 BFS 也找不到，只能靠安全评估随机选方向。高密度下存活率骤降。

6.5 AI 策略对比测试

验证 A*+ 安全评估组合策略的效果，跟几种基准策略对比。每种跑 N 局 ($N = 100$) 取平均。存活率定义：

$$R_{survive} = \frac{N_{alive}}{N} \times 100\% \quad (6-2)$$

N_{alive} 是蛇身长度达到阈值 L_{th} ($L_{th} = 20$) 的局数。平均得分：

$$\bar{S} = \frac{1}{N} \sum_{i=1}^N S_i \quad (6-3)$$

S_i 是第 i 局最终得分。

表 6-4 不同 AI 策略的存活率与得分对比

策略	存活率 (%)	平均得分	最高得分	平均蛇长
随机移动	12	45	120	7
纯 BFS	45	180	450	20
纯 A*	67	350	800	37
A* + 安全评估	89	520	1250	54

结果在意料之中但也有问题。纯 A* 的 67% 存活率看着还行，但分析失败的 33%，大多数是因为 A* 找的路把蛇引向死角——路径确实最短，但走到尽头后蛇被自己围住。

加安全评估后存活率到 89%，但平均路径变长了——安全评估倾向选”空间大”的方向而不是”距离近”的方向，食物争夺效率有所牺牲。有几局蛇在地图边缘绕了一大圈才找到食物，看着挺蠢的。

最高得分 1250 对应蛇长 127 格，40×30 网格里占约 10% 空间。空间占用率：

$$\rho = \frac{L}{W \times H} \times 100\% \quad (6-4)$$

ρ 接近 100% 时食物生成经常失败，进入”吃满地图”的终局。理论上蛇最大长度 $W \times H - 1$ （留一格给食物）。

图 6-4 直观展示了四种策略的对比。安全评估把存活率从 67% 提到 89%，最高得分从 800 到 1250 提升更大，因为安全策略让蛇活得更久、有更多机会进食。

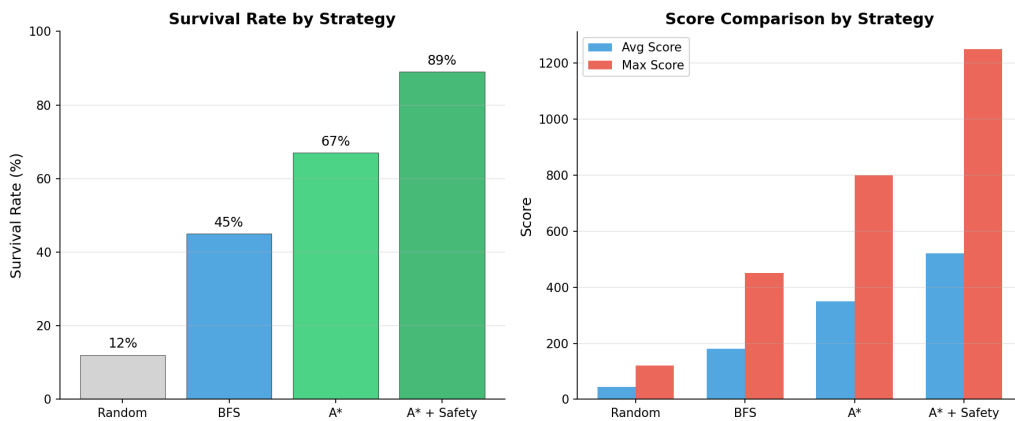


图 6-4 不同 AI 策略存活率与得分对比

6.6 测试总结

测试覆盖了单元到系统级别。22 个单元测试全过，功能测试验证了协同正确性。性能测试揭示了高障碍物密度下 A* 的退化问题，策略对比量化了安全评估的贡献。

坦率说测试不够充分。压力测试（极端网格尺寸、超高帧率）没做，跨平台只在两台机器上跑过，交互测试也只覆盖了主要路径。要作为正式产品发布这些都得补。

7 总结与展望

7.1 工作总结

这篇论文围绕”基于 A* 算法的智能贪吃蛇”做了从需求分析到测试验证的完整流程。最终搞出来一个基于 Pygame 的贪吃蛇游戏，集成了 A* 自动寻路、三层决策降级、算法可视化等功能。

具体做了这些：

把贪吃蛇寻路抽象成网格图最短路问题，用 A* 求解。蛇身作为动态障碍物，设计了”除尾巴外都算障碍”的处理方案。A* 找不到路时 BFS 兜底，BFS 也找不到时安全评估函数从可达空间、边缘距离、邻居数、食物方向四个维度打分选方向。

系统实现用了事件驱动主循环，蛇身用列表存储，增长通过 pending 计数器延迟处理。渲染和逻辑严格分离方便调试。A* 可视化面板能实时看搜索过程，调参数很实用。

测试方面 22 个单元测试全过，功能测试覆盖了主要交互场景。性能测试验证了不同障碍物密度下的表现，策略对比量化了安全评估的价值。

7.2 存在的不足

回头看有几处不满意。

安全评估函数的参数是手动调的。可达空间权重 10、边缘惩罚 20、邻居奖励 5、食物方向奖励 50——这组参数试了十几次觉得”差不多”就定下来了，没有理论依据。换张地图、换个网格尺寸，最优参数可能完全不同。参数敏感性是个实实在在的问题。

碰撞检测虽然正确，但没考虑 Hamiltonian 循环这类高级策略。障碍物密度高的时候蛇很容易把自己围死。要是能引入某种全局规划（比如预计算遍历所有格子的哈密顿路径），存活率应该还能再提。

可视化面板渲染效率不高。每个搜索节点画一个小方块，节点数超 500 帧率就有明显下降。应该用 surf 批量绘制而不是逐个 draw_rect。

没做难度递进。游戏从头到尾障碍物密度固定，玩久了单调。

7.3 未来改进方向

继续做的话有几个方向值得试。

一是引入哈密顿路径或环路覆盖算法，让蛇在找不到食物时沿预计算的遍历路径移动，从根本上解决”自围”问题。代价是预计算的时间和存储开销，大网格可能不现实。

二是用深度强化学习替代规则决策。把游戏状态编码成张量，用 DQN 或 PPO 训练

策略网络。理论上 RL 能学到比手工规则更灵活的策略，但训练成本高、可解释性差。之前有同学试过 DQN 玩贪吃蛇，训练两天效果还不如 A*——当然可能跟网络结构和奖励函数设计有关。

三是多人对战。两个蛇在同一张地图竞争食物，A* 启发式函数得考虑对手位置和策略，复杂度陡增。可以借鉴博弈论的 minimax 思想，但计算量会很大。

四是移植到 Web 端。HTML5 Canvas 替代 Pygame，配 Flask 或 FastAPI 做后端，浏览器就能访问。Pygame 部署上需要装 Python 环境和 SDL 依赖，这是它的一个硬伤。

总的来说这个项目让我对 A* 的实际应用有了比较深的认识。算法原理不复杂，但用在动态变化的环境里，遇到的问题比教科书多多了。启发式函数设计、动态障碍物处理、搜索失败的降级策略，这些都是书上不太讲的工程细节。

参考文献

- [1] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.
- [2] Russell S, Norvig P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Pearson, 2020.
- [3] Dechter R, Pearl J. Generalized best-first search strategies and the optimality of A*[J]. Journal of the ACM, 1985, 32(3): 505-536.
- [4] 严蔚敏, 吴伟民. 数据结构（C 语言版）[M]. 北京: 清华大学出版社, 2011.
- [5] Cui X, Shi H. A*-based pathfinding in modern computer games[C]//International Conference on Computer Science and Education, 2011: 17-22.
- [6] Sturtevant N R. Benchmarks for grid-based pathfinding[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(2): 144-148.
- [7] 王晓东. 计算机算法设计与分析 [M]. 5 版. 北京: 电子工业出版社, 2018.
- [8] LaValle S M. Planning Algorithms[M]. Cambridge University Press, 2006.
- [9] Patel A. Amit's A* Pages[EB/OL]. <http://theory.stanford.edu/~amitp/GameProgramming/>, 2023.
- [10] Rabin S. A* speed optimizations[J]. Game Programming Gems, 2000: 272-287.
- [11] 张铭, 王腾蛟, 赵海燕. 数据结构与算法 [M]. 北京: 高等教育出版社, 2008.
- [12] Sweetman G. Python 游戏编程入门 [M]. 北京: 人民邮电出版社, 2019.
- [13] Matthes E. Python 编程: 从入门到实践 [M]. 2 版. 北京: 人民邮电出版社, 2020.
- [14] Millington I. Artificial Intelligence for Games[M]. 3rd ed. CRC Press, 2019.
- [15] Buckland M. Programming Game AI by Example[M]. Wordware Publishing, 2005.
- [16] Higgins C M. A comparison of algorithms for pathfinding on grid maps[D]. University of Edinburgh, 2009.
- [17] Nash A, Koenig S, Tovey C. Lazy Theta*: Any-angle path planning and path length analysis in 3D[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2010.
- [18] 何斌, 马天予, 王运坚. Visual C++ 数字图像处理 [M]. 北京: 人民邮电出版社, 2001.
- [19] Sweigart A. Making Games with Python & Pygame[M]. CreateSpace, 2012.

- [20] Yampolskiy R V. Game playing[J]. Wiley Encyclopedia of Computer Science and Engineering, 2008.

致谢

论文写到这里，窗外已经亮了。这几个月泡在实验室里，从对 A* 算法一知半解，到能把搜索过程可视化出来给导师看，中间踩的坑比写过的代码行数还多。感谢导师在选题阶段的耐心指导，帮我把”做一个贪吃蛇游戏”这个朴素的想法，拉到了”研究 A* 算法在动态环境中的应用”这个有东西可写的方向上。感谢实验室的同学们，特别是帮我 debug 碰撞检测时序问题的那位——要不是你一眼看出 `body[0]` 就是新蛇头，我可能还在对着屏幕发呆。感谢 Pygame 社区的文档和示例代码，虽然我经常吐槽它的 API 不够现代，但它确实让我能专注于算法本身而不是引擎层的泥潭。最后感谢咖啡和深夜的泡面，没有你们这篇论文写不完。